# Home Assignment 3

## Виртуальная машина, JupyterLab и LLM-проекты в Yandex Cloud

Домашнее задание направлено на практическое освоение облачной инфраструктуры, удалённой разработки, JupyterLab и подключения языковых моделей через API.

**Основной результат: рабочий прототип, в котором виртуальная машина, Python/Jupyter и LLM используются вместе для решения прикладной задачи.**

*Можно потерять деньги, гоняясь за девчонкой, но никогда не потеряешь девчонку, гоняясь за деньгами. С нейросетями та же история: можно потратить жизнь, бегая за трендами, но, если ты научился превращать ИИ в навык, продукт и деньги — тренды уже начинают бегать за тобой.*

## 1. Общая постановка задания

Для выполнения данного домашнего задания необходимо поднять виртуальную машину в Yandex Cloud, подключиться к ней по SSH, установить Python-окружение и JupyterLab, открыть Jupyter через localhost и подключить LLM через Yandex AI Studio. Подробная техническая инструкция по развёртыванию находится в файле yandex_cloud_vm_jupyter_llm_lecture.md.

В рамках этой домашней работы важно не просто повторить команды установки, а показать, что студент понимает архитектуру решения: где работает виртуальная машина, где запускается Jupyter, где хранится код, как передаются запросы в LLM, как возвращается ответ модели и как этот ответ используется в проекте.

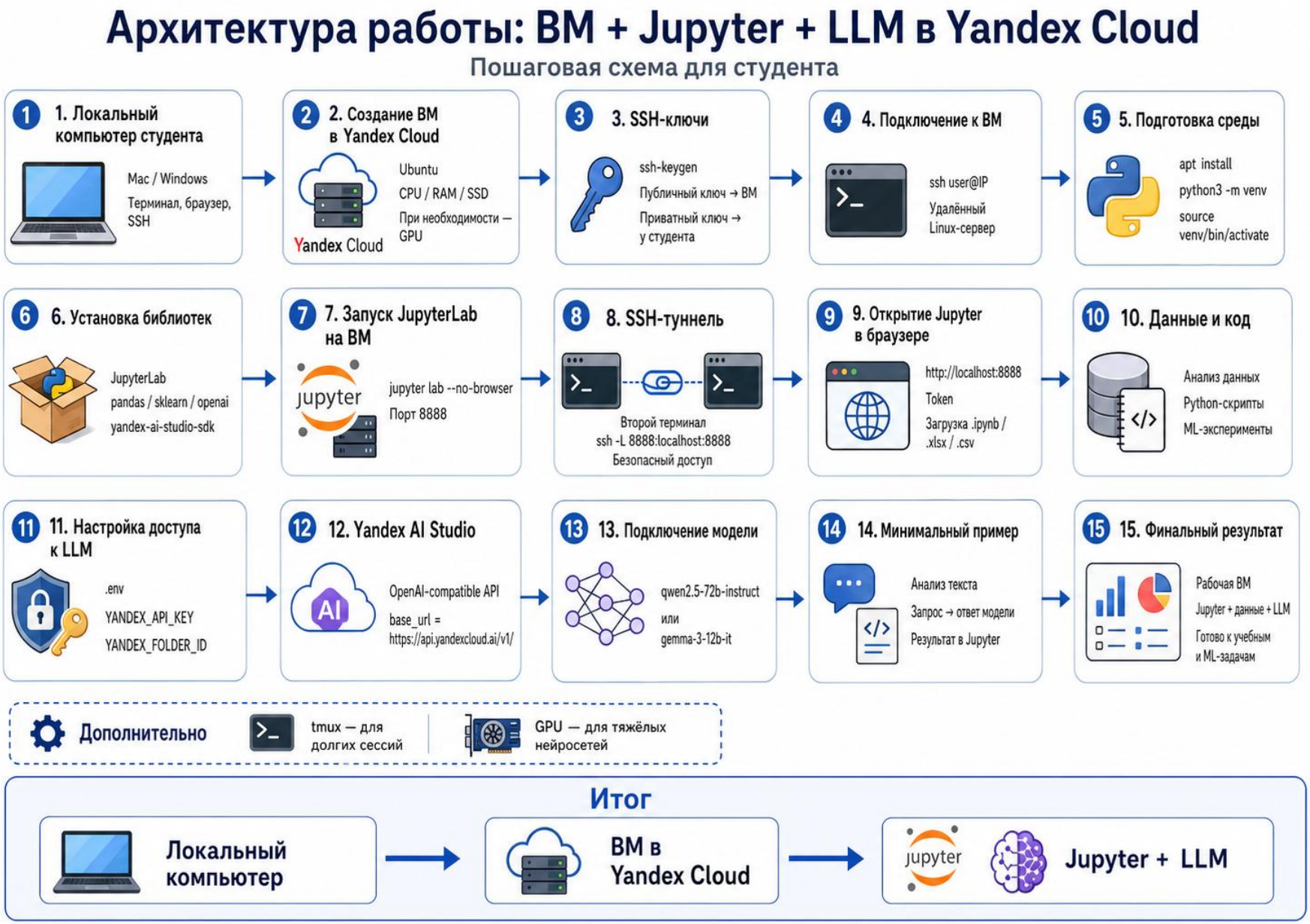
Итоговая работа должна быть маленьким, но воспроизводимым AI-проектом. В нём должны быть видны техническая часть (ВМ, окружение, запуск), логика обращения к языковой модели и прикладной результат: таблица, отчёт, бот, сайт, workflow или другой работающий прототип.

## 2. Обязательный минимум для всех

Чтобы работа была принята к проверке, необходимо выполнить обязательный технический минимум:

1. Создать виртуальную машину в Yandex Cloud или использовать уже созданную учебную ВМ.
2. Подключиться к ВМ по SSH и продемонстрировать работоспособность подключения.
3. Создать проектную папку и виртуальное окружение Python (venv).
4. Установить JupyterLab и основные библиотеки для анализа данных и работы с LLM.
5. Запустить JupyterLab на ВМ и открыть его на локальном компьютере через SSH-туннель и localhost.
6. Создать файл .env или .env.example и показать корректную логику хранения ключей без публикации настоящих API-ключей.
7. Подключить LLM через Yandex AI Studio API. Рекомендуемые модели: qwen2.5-72b-instruct или gemma-3-12b-it.
8. Выполнить тестовый запрос к LLM из Jupyter Notebook или Python-скрипта.
9. Выбрать один проект из списка ниже или предложить собственный проект сопоставимой сложности.
10. Подготовить краткий отчёт, скриншоты и исходные файлы проекта.

**Важно.** Инструкцию по созданию ВМ, установке JupyterLab, запуску через localhost и подключению LLM не нужно полностью переписывать в отчёте. Достаточно сослаться на файл yandex_cloud_vm_jupyter_llm_lecture.md и показать, что у вас всё реально работает.




*Общая архитектура выполнения задания: локальный компьютер, ВМ, JupyterLab и LLM*

## 3. Выбор проектной темы

После выполнения обязательного минимума студент выбирает один проект. Можно выбрать тему из предложенного списка или согласовать собственную тему. Собственная тема должна быть не слабее предложенных вариантов: в ней обязательно должны присутствовать ВМ, Jupyter или веб/backend-среда, обращение к LLM и видимый практический результат.

Проекты отличаются уровнем сложности. Минимальный уровень показывает, что связка ВМ + Jupyter/Python + LLM работает. Хорошая работа добавляет обработку данных, интерфейс или логирование. Цель максимум предполагает более цельный продукт: сайт, бот, workflow, RAG или генератор отчётов.

### Допустимое использование vibe-coding

При разработке проектов можно использовать нейросети как помощников по коду, архитектуре и отладке. Это не запрещается. Более того, в реальной разработке AI-инструменты часто используются именно так: как ускоритель написания кода, объяснения ошибок и поиска решений.

- Проверяйте код, который предлагает нейросеть: запускайте его, смотрите ошибки, исправляйте зависимости и понимайте, что делает каждая часть.
- Не публикуйте настоящие API-ключи в GitHub, отчёте, скриншотах и ноутбуках.
- В отчёте объясняйте, какая часть проекта работает на ВМ, где запускается LLM-запрос, какие данные передаются в модель и какой результат возвращается.
- Если проект связан с сообщениями людей, используйте режим черновиков или явное подтверждение пользователя. Не делайте скрытую подмену реального человека без контроля.
- Главное требование: проект должен быть воспроизводимым. Другой человек должен понять, как запустить вашу работу по README и .env.example.

## 4. Список проектных вариантов

| № | Проект | Суть | Сложность |
|---|---|---|---|
| 1 | AI-помощник для анализа данных в Jupyter | Jupyter + pandas/sklearn + LLM-интерпретация результатов | низкая/средняя |
| 2 | Анализатор текстов через LLM | Разметка текстов: тема, тональность, категория, резюме | низкая |
| 3 | AI-помощник для Excel / CSV | Загрузка таблицы, анализ, графики, LLM-выводы | средняя |
| 4 | AI-код-ревьюер | Проверка Python-кода, объяснение ошибок и рекомендации | средняя |
| 5 | Telegram-бот с LLM | Бот получает сообщение, отправляет его в LLM и отвечает пользователю | средняя/высокая |
| 6 | Сайт-анализатор текста | Streamlit/FastAPI-интерфейс для анализа текста через LLM | средняя |
| 7 | n8n + LLM workflow | Визуальная автоматизация: webhook/Telegram/Sheets -> LLM -> результат | средняя |
| 8 | mini-RAG по учебным материалам | Ответы по документам курса на основе retrieval и LLM | высокая |
| 9 | Генератор отчёта по ML-модели | ML pipeline -> метрики -> LLM-отчёт | средняя/высокая |
| 10 | Telegram-автопомощник / style assistant | Сложный Telegram-проект с буферизацией, стилем, логированием и risk-check | высокая |

# Проект 1. AI-помощник для анализа данных в Jupyter

**Описание проекта.** Это базовый и наиболее реалистичный вариант проекта. Студент выполняет обычный анализ данных в Jupyter Notebook, но добавляет внутрь ноутбука функцию обращения к LLM. Нейросеть выступает не заменой анализа, а помощником: помогает объяснить структуру данных, сформулировать гипотезы, интерпретировать графики и подготовить текстовые выводы.

**Цель проекта.** Показать, как LLM может быть встроена в рабочее пространство аналитика данных и использоваться прямо внутри Jupyter, где уже выполняются расчёты, строятся графики и обучаются модели.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Студент открывает Jupyter на ВМ, загружает датасет, выполняет EDA и получает промежуточные численные результаты. После этого он формирует prompt, куда передаёт не всю таблицу, а краткое описание: названия колонок, статистики, корреляции, результаты графиков или метрики модели. LLM возвращает текстовую интерпретацию и рекомендации. | LLM помогает формулировать аналитические выводы, объяснять смысл показателей, предлагать гипотезы и писать текст отчёта. Расчёты выполняет Python; LLM не должна придумывать числа. |

## Основные задачи реализации

- Подготовить небольшой датасет в CSV/XLSX или взять открытый датасет.
- Загрузить данные в pandas и выполнить первичный анализ.
- Построить несколько таблиц, описательных статистик и графиков.
- Создать функцию обращения к LLM, например ask_llm(prompt).
- Использовать LLM минимум для трёх задач: объяснение данных, интерпретация результата, итоговый вывод.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| ВМ + Jupyter + датасет + минимум 3 осмысленных обращения к LLM: описание данных, интерпретация результата, итоговый вывод. | Добавить EDA: таблицы, описательные статистики, графики, корреляции/группировки. LLM должна объяснять результаты, а не заменять расчёты. | Сформировать полуавтоматический отчёт: Python считает статистики и строит графики, LLM генерирует структурированные выводы и рекомендации. | Файл .ipynb, скриншоты Jupyter и VM, исходные данные или ссылка на них, краткий отчёт с указанием мест, где использовалась LLM. |

## Ключевое взаимодействие ВМ и LLM

- Python и Jupyter работают на ВМ.
- Данные загружаются в Jupyter и обрабатываются через pandas/sklearn.
- В LLM передаются только сформированные контексты: описание данных, метрики, результаты графиков, вопросы пользователя.
- Ответ модели сохраняется в ноутбуке как часть аналитического результата.

## Методические замечания

- Не отправляйте в LLM всю большую таблицу целиком: лучше передавать агрегаты, статистики и краткие фрагменты.
- Отдельно покажите ячейки, где Python считает результат, и ячейки, где LLM помогает его объяснить.

*Архитектура проекта: Проект 1. AI-помощник для анализа данных в Jupyter*

# Проект 2. Анализатор текстов через LLM

**Описание проекта.** Проект предназначен для автоматической обработки набора текстов: отзывов, обращений, комментариев, новостей, сообщений поддержки или учебных ответов. Система должна принимать тексты, отправлять их в LLM и возвращать структурированную разметку.

**Цель проекта.** Научиться использовать языковую модель как инструмент интеллектуальной разметки текстовых данных: определять тему, тональность, категорию, краткое резюме и необходимость реакции.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| На вход подаётся таблица с текстами. Python проходит по строкам, формирует для каждой строки prompt и отправляет его в LLM. Ответ модели парсится, превращается в новые колонки DataFrame и затем используется для отчёта или визуализации. | LLM выполняет смысловую классификацию текста: определяет тему, тональность, срочность, резюме и признак action_required. Python отвечает за загрузку, циклы, таблицы, сохранение и визуализацию. |

## Основные задачи реализации

- Собрать таблицу с текстами или загрузить CSV/Excel с текстовой колонкой.
- Сделать очистку и нормализацию текста.
- Разработать prompt-шаблон для анализа одного текста или батча.
- Получить от LLM структурированный ответ, желательно в JSON-формате.
- Собрать итоговую таблицу и построить простую сводку по категориям/тональности.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Таблица из 10–20 текстов, функция отправки текста в LLM, итоговая таблица с результатами анализа. | Добавить пакетную обработку, контроль формата ответа, сохранение результата в CSV/XLSX и простые визуализации. | Сделать полноценный pipeline: очистка текста, prompt templates, JSON-ответ от LLM, валидация, отчёт по категориям и тональности. | Ноутбук или скрипт, таблица входных текстов, таблица результатов, скриншот успешного LLM-запроса, краткие выводы. |

## Ключевое взаимодействие ВМ и LLM

- ВМ используется как среда выполнения Python/Jupyter.
- LLM получает структурированный prompt для каждого текста или батча текстов.
- Результаты возвращаются в DataFrame и используются для отчёта или визуализации.
- Желательно предусмотреть обработку ошибок и повторный запрос при некорректном JSON.

## Методические замечания

- Для минимального уровня можно использовать искусственную таблицу из 10–20 текстов.
- Для хорошей работы желательно показать, как результат LLM превращается в новые колонки таблицы.

*Архитектура проекта: Проект 2. Анализатор текстов через LLM*

# Проект 3. AI-помощник для Excel / CSV

**Описание проекта.** Проект имитирует прикладной инструмент для пользователя, который загружает таблицу и хочет быстро понять, что в ней происходит. Система должна не просто открыть Excel/CSV, а выполнить первичный анализ и сформировать понятные текстовые выводы.

**Цель проекта.** Показать, как LLM может помогать при работе с табличными данными: объяснять колонки, интерпретировать статистику, находить проблемы качества данных и формировать краткую аналитическую записку.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Пользователь загружает файл, Python читает его через pandas/openpyxl, строит статистики и графики. Затем в LLM отправляется короткое аналитическое резюме: какие колонки есть, какие пропуски, какие зависимости заметны, какие группы отличаются. LLM возвращает текстовые выводы. | LLM не считает таблицу вместо pandas. Она получает уже рассчитанный контекст и переводит его в понятный аналитический текст: выводы, объяснения, возможные причины, рекомендации и ограничения. |

## Основные задачи реализации

- Реализовать загрузку CSV/XLSX через Jupyter, Streamlit или простую функцию.
- Проверить типы данных, пропуски, дубликаты и базовые статистики.
- Построить минимум 2–3 графика или сводные таблицы.
- Сформировать текстовый контекст для LLM на основе рассчитанных значений.
- Получить от LLM объяснение результатов и рекомендации по дальнейшему анализу.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Загрузить CSV/XLSX, посчитать описательные статистики, построить минимум один график и получить LLM-вывод по результатам. | Добавить автоматическую проверку пропусков, типов данных, группировки и экспорт результата. | Сделать мини-приложение или расширенный ноутбук, где пользователь задаёт вопросы по таблице, а LLM отвечает на основе рассчитанных статистик. | Excel/CSV-файл, ноутбук или приложение, итоговая таблица/графики, текстовые выводы LLM, .env.example. |

## Ключевое взаимодействие ВМ и LLM

- Файл обрабатывается на ВМ в Python.
- Статистику считает pandas, а не LLM.
- В LLM передаётся краткий статистический контекст: размер таблицы, названия колонок, пропуски, основные показатели, графики/метрики.
- Ответ LLM должен быть сохранён как пояснение к результатам анализа.

## Методические замечания

- Хорошая практика: сохранять итоговый отчёт в Markdown или HTML.
- Можно добавить кнопку/ячейку “сформировать вывод”, которая отправляет агрегированный контекст в LLM.

*Архитектура проекта: Проект 3. AI-помощник для Excel / CSV*

# Проект 4. AI-код-ревьюер

**Описание проекта.** Проект представляет собой ассистента для проверки Python-кода. Пользователь вставляет фрагмент кода или traceback, а система возвращает объяснение, возможные ошибки, замечания по стилю и рекомендации по исправлению.

**Цель проекта.** Научиться использовать LLM как помощника по программированию, но с контролируемым prompt-форматом и понятной структурой ответа.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Студент или пользователь вставляет код. Система может выполнить базовую проверку синтаксиса или просто передать код в LLM. Модель анализирует фрагмент и возвращает ревью. Результат показывается как отчёт, где отдельно описаны ошибки, стиль и возможная исправленная версия. | LLM объясняет код и предлагает исправления. Она особенно полезна для интерпретации traceback и формулировки понятных рекомендаций. При этом студент должен проверить, что предложенный код действительно работает. |

## Основные задачи реализации

- Реализовать ввод кода в Jupyter, текстовом файле или веб-форме.
- Сформировать prompt, который просит модель отвечать по разделам.
- Передавать в LLM код, ошибку, контекст задания и ожидаемый формат ответа.
- Получать структурированный ответ: смысл кода, ошибки, стиль, оптимизация, исправления.
- Сохранить результат как Markdown-отчёт или ячейку ноутбука.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Функция в Jupyter: пользователь вставляет код, LLM возвращает объяснение и рекомендации. | Добавить обработку traceback, разделы отчёта: смысл кода, ошибки, стиль, оптимизация, исправленная версия. | Сделать сайт или форму, где пользователь вставляет код, получает структурированное ревью и может скачать Markdown-отчёт. | Примеры проверенного кода, ответы LLM, отчёт с рекомендациями, исходный код проекта. |

## Ключевое взаимодействие ВМ и LLM

- Код/ошибка вводится в Jupyter или веб-форму на ВМ.
- Prompt Builder формирует структурированный запрос для LLM.
- LLM возвращает ответ по разделам, а проект форматирует его в понятный отчёт.
- Важно не запускать небезопасный чужой код без изоляции. Для минимального проекта достаточно анализа текста кода без выполнения.

## Методические замечания

- Если вы запускаете чужой код, делайте это только в безопасной изолированной среде.
- В отчёте обязательно покажите пример: входной код -> ответ ревьюера -> исправленный вариант.

*Архитектура проекта: Проект 4. AI-код-ревьюер*

# Проект 5. Telegram-бот с LLM

**Описание проекта.** Проект предполагает создание Telegram-бота, который принимает сообщение пользователя, отправляет его в LLM через backend на ВМ и возвращает ответ в Telegram. Это уже продуктовый сценарий: пользователь взаимодействует не с ноутбуком, а с ботом.

**Цель проекта.** Показать, как ВМ может использоваться как backend-сервер для бота, а LLM — как интеллектуальное ядро, которое генерирует ответы.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Пользователь пишет сообщение в Telegram-бота. Telegram Bot API передаёт его Python-приложению на ВМ. Backend формирует контекст и prompt, вызывает LLM, форматирует ответ и отправляет его обратно пользователю. | LLM генерирует содержательный ответ на вопрос пользователя. В зависимости от выбранной темы бот может быть учебным ассистентом, справочником, помощником по анализу данных или текстовым консультантом. |

## Основные задачи реализации

- Создать Telegram-бота через BotFather и получить токен.
- Поднять Python-приложение на ВМ с aiogram или pyTelegramBotAPI.
- Реализовать обработку входящих сообщений.
- Сформировать prompt и отправить его в Yandex AI Studio.
- Вернуть ответ LLM пользователю и сохранить пример логов.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Простой бот: вопрос в Telegram -> LLM -> ответ в Telegram. | Добавить команды, системный prompt, логирование сообщений, обработку ошибок и ограничение длины запроса. | Сделать полноценного учебного ассистента: разные режимы ответа, история диалога, сохранение логов, деплой через systemd/Docker. | Скриншоты переписки с ботом, код backend, .env.example, описание запуска, логи или пример работы. |

## Ключевое взаимодействие ВМ и LLM

- Backend бота работает на ВМ.
- Telegram Bot API передаёт сообщения в Python-приложение.
- Приложение формирует prompt и отправляет его в Yandex AI Studio.
- Ответ LLM форматируется и отправляется обратно пользователю в Telegram.
- Настоящие токены Telegram и API-ключи нельзя публиковать.

## Методические замечания

- Для зачёта достаточно polling-режима, если webhook сложен в настройке.
- Обязательно покажите, что бот работает именно через LLM, а не возвращает заранее заготовленный текст.

*Архитектура проекта: Проект 5. Telegram-бот с LLM*

# Проект 6. Сайт-анализатор текста

**Описание проекта.** Проект должен выглядеть как простой веб-сервис: пользователь открывает страницу, вводит текст или загружает файл, нажимает кнопку анализа и получает результат от LLM. Это хороший вариант для тех, кто хочет показать не только ноутбук, но и пользовательский интерфейс.

**Цель проекта.** Создать минимальное веб-приложение, в котором LLM используется как сервис анализа текста: резюме, ключевые слова, категория, тональность, рекомендации.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Пользователь заходит на сайт, вставляет текст и выбирает тип анализа. Backend на ВМ принимает текст, формирует prompt и отправляет запрос в LLM. Ответ модели парсится и показывается в виде карточек: резюме, тональность, ключевые слова, рекомендации. | LLM выполняет смысловую обработку текста и возвращает результат в удобном формате. Веб-интерфейс только собирает ввод, запускает анализ и показывает результат. |

## Основные задачи реализации

- Выбрать интерфейс: Streamlit, Gradio или FastAPI + простая HTML-страница.
- Реализовать поле ввода текста или загрузку файла.
- Провести базовую валидацию и предобработку текста.
- Отправить prompt в Yandex AI Studio и получить ответ.
- Отобразить результат на странице в структурированном виде.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Мини-сайт на Streamlit: поле ввода текста -> кнопка анализа -> ответ LLM на странице. | Добавить загрузку TXT/CSV, выбор задач анализа, структурированный вывод и экспорт результата. | Сделать полноценный сервис с backend, обработкой ошибок, кэшем запросов и аккуратным интерфейсом. | Файл app.py или проект, скриншот интерфейса, пример входного текста, результат LLM, инструкция запуска. |

## Ключевое взаимодействие ВМ и LLM

- Сайт или Streamlit-приложение запускается на ВМ.
- Доступ можно делать через SSH-туннель или временно через открытый порт при соблюдении безопасности.
- Backend получает текст, формирует prompt, отправляет запрос в LLM и отображает результат пользователю.
- Желательно добавить ограничения длины текста и обработку таймаутов.

## Методические замечания

- Streamlit — самый простой вариант для учебного проекта.
- Если открываете порт наружу, следите за безопасностью и не публикуйте ключи.

*Архитектура проекта: Проект 6. Сайт-анализатор текста*

# Проект 7. n8n + LLM workflow

**Описание проекта.** Проект ориентирован на визуальную автоматизацию. Вместо большого Python-кода студент собирает цепочку в n8n: событие приходит из формы, Telegram, webhook или Google Sheets, затем текст отправляется в LLM, а результат сохраняется или отправляется дальше.

**Цель проекта.** Показать, что LLM можно встраивать не только в Python-ноутбуки, но и в автоматизированные рабочие процессы: обработка события, вызов модели, ветвление и сохранение результата.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Во внешний источник приходит событие: сообщение, форма или строка таблицы. n8n запускает workflow, нормализует данные, отправляет prompt в LLM, затем по результату выполняет действие: сохраняет ответ, отправляет уведомление или маршрутизирует задачу. | LLM выступает как интеллектуальный узел workflow: классифицирует, резюмирует, оценивает риск, формирует ответ или принимает решение для дальнейшего ветвления. |

## Основные задачи реализации

- Поднять n8n на ВМ или использовать доступный n8n-инстанс.
- Создать триггер: webhook, форма, Telegram или новая строка в Google Sheets.
- Подготовить текст и prompt для LLM.
- Отправить запрос в Yandex AI Studio через HTTP/API node.
- Сохранить результат в Google Sheets, отправить в Telegram или вывести как ответ webhook.

## Уровни выполнения

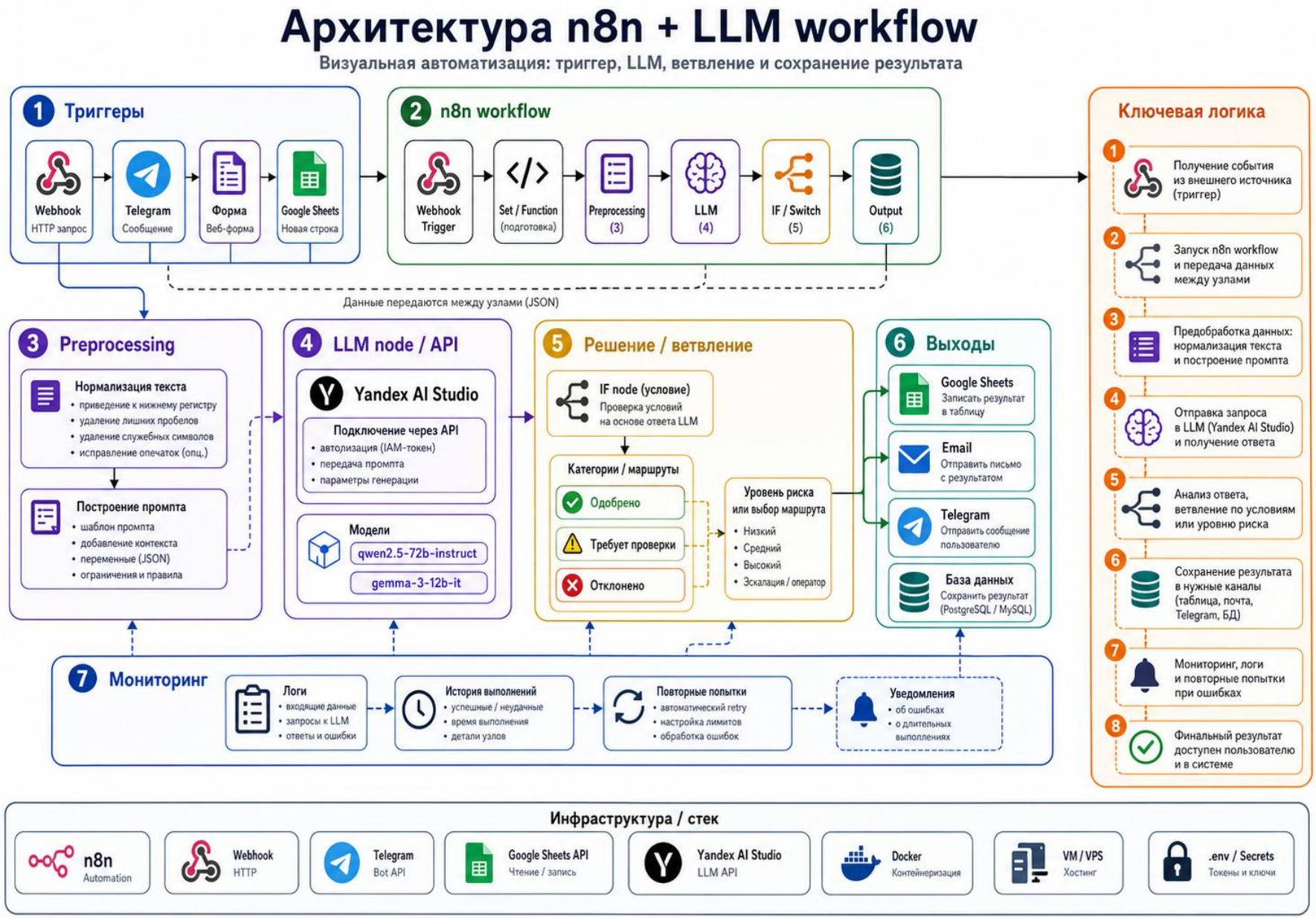
| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| n8n workflow: Webhook -> подготовка текста -> LLM-запрос -> вывод результата. | Добавить ветвление IF/Switch, сохранение результата в Google Sheets или отправку в Telegram. | Сделать устойчивый процесс с логированием, повторными попытками, уведомлениями и несколькими каналами вывода. | Экспорт workflow или скриншоты узлов, описание логики, пример входа/выхода, результат работы LLM. |

## Ключевое взаимодействие ВМ и LLM

- n8n может быть поднят на ВМ или использоваться как отдельный сервис.
- LLM вызывается через HTTP/API node или через скриптовый узел.
- Данные между узлами передаются в JSON.
- Студент должен показать схему workflow и пример успешного выполнения.

## Методические замечания

- Даже если n8n используется в облаке, объясните, где находится LLM-запрос и как хранятся ключи.
- Важно показать не только красивые узлы, но и реальный запуск workflow.




*Архитектура проекта: Проект 7. n8n + LLM workflow*

# Проект 8. mini-RAG по учебным материалам

**Описание проекта.** Проект создаёт маленького ассистента, который отвечает на вопросы не просто из “общих знаний” LLM, а по загруженным учебным материалам: лекциям, конспектам, PDF/DOCX/TXT-файлам. Это приближает проект к реальным системам поиска по корпоративным документам.

**Цель проекта.** Освоить принцип RAG: сначала найти релевантные фрагменты документов, затем передать их в LLM как контекст и получить ответ с опорой на источники.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Пользователь задаёт вопрос по учебному материалу. Retriever находит top-k релевантных фрагментов, Prompt Builder объединяет вопрос и контекст, после чего LLM формирует ответ. Желательно возвращать не только ответ, но и ссылки/названия использованных источников. | LLM отвечает на вопрос на основе переданного контекста. Она не должна заменять поиск: сначала Python находит источники, затем LLM формулирует понятный ответ. |

## Основные задачи реализации

- Загрузить несколько учебных материалов или текстовых файлов.
- Извлечь текст и разбить его на фрагменты.
- Реализовать поиск релевантных фрагментов: keyword search, TF-IDF или embeddings.
- Сформировать prompt: вопрос пользователя + найденный контекст.
- Получить ответ LLM и показать, какие фрагменты были использованы.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Загрузить 2–3 текстовых материала, реализовать поиск релевантных фрагментов через TF-IDF или простой keyword search, передать фрагменты в LLM. | Добавить chunking, top-k retrieval, ссылки на источники/страницы, сохранение истории вопросов. | Сделать mini-RAG с embeddings и FAISS/Chroma, интерфейсом вопросов и ответами с источниками. | Материалы или их фрагменты, ноутбук/приложение, примеры вопросов, ответы с источниками, описание retrieval-логики. |

## Ключевое взаимодействие ВМ и LLM

- Материалы хранятся и обрабатываются на ВМ.
- Retriever ищет релевантные фрагменты.
- В LLM передаётся не весь документ, а вопрос + найденный контекст.
- Ответ должен быть связан с источниками, а не просто сгенерирован из общих знаний модели.

## Методические замечания

- Для минимального уровня embeddings не обязательны: можно начать с TF-IDF.
- Главное — показать связку “поиск контекста -> prompt -> ответ LLM”.

*Архитектура проекта: Проект 8. mini-RAG по учебным материалам*

# Проект 9. Генератор отчёта по ML-модели

**Описание проекта.** Проект соединяет классический ML и LLM. Студент обучает модель, рассчитывает метрики, строит графики, а затем передаёт структурированный summary в LLM, чтобы получить аккуратный аналитический отчёт.

**Цель проекта.** Показать правильное разделение ролей: Python обучает модель и считает метрики, а LLM помогает оформить результаты в понятный отчёт с интерпретацией и рекомендациями.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Python обучает модель и сохраняет результаты: метрики, графики, параметры, важность признаков. Summary Builder превращает это в компактный контекст. LLM получает контекст и генерирует отчёт с разделами: цель, методология, метрики, интерпретация, выводы, рекомендации. | LLM оформляет аналитический текст и помогает объяснить результаты. Численные значения, графики и метрики должны быть заранее рассчитаны Python-кодом. |

## Основные задачи реализации

- Выбрать задачу классификации или регрессии и подготовить датасет.
- Построить ML pipeline: preprocessing, train/test split, обучение модели.
- Посчитать метрики и построить графики.
- Собрать структурированный JSON/словарь с результатами.
- Передать summary в LLM и получить Markdown-отчёт.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Обучить модель sklearn, посчитать метрики, передать метрики и краткий контекст в LLM, получить текстовый отчёт. | Добавить графики, confusion matrix/feature importance, структурированный JSON-свод и Markdown-отчёт. | Сделать генератор отчёта с несколькими моделями, сравнением, экспортом в HTML/PDF/Markdown и сохранением артефактов. | Ноутбук с ML-моделью, метрики, графики, LLM-отчёт, сохранённые артефакты или Markdown-файл. |

## Ключевое взаимодействие ВМ и LLM

- ML-обучение и расчёт метрик выполняются в Python на ВМ.
- LLM не должна придумывать метрики: ей передаются рассчитанные значения.
- Prompt должен содержать задачу, данные, модель, метрики, ограничения и желаемую структуру отчёта.
- Результат LLM используется как аналитический текст, а не как единственный источник численных данных.

## Методические замечания

- Хороший вариант — сравнить 2 модели и попросить LLM объяснить, какая лучше и почему.
- Можно добавить экспорт отчёта в .md или .html.

*Архитектура проекта: Проект 9. Генератор отчёта по ML-модели*

# Проект 10. Telegram-автопомощник / style assistant

**Описание проекта.** Это сложный вариант повышенной сложности. Проект моделирует персонального Telegram-ассистента, который анализирует входящие сообщения, буферизует пачки, формирует контекст, генерирует черновик ответа в заданном стиле, логирует действия и применяет risk-check. Для учебной сдачи не требуется делать скрытый автопилот: достаточно контролируемого MVP с черновиками и ручным подтверждением.

**Цель проекта.** Показать, как можно построить сложную LLM-систему с несколькими сервисами: Telegram, буфер сообщений, база логов, профиль стиля, prompt builder, LLM, фильтр риска и отправка/черновик ответа.

| Что должно происходить в проекте | Как здесь используется LLM |
|---|---|
| Входящие сообщения попадают в буфер, объединяются в единый контекст, записываются в лог, затем Prompt Builder добавляет стиль, историю и правила безопасности. LLM возвращает JSON с risk_level, should_autosend и answer. Далее система либо показывает черновик, либо блокирует автодействие, либо в безопасном варианте отправляет ответ. | LLM генерирует черновик ответа в заданном стиле и помогает классифицировать риск. Однако финальная отправка должна быть контролируемой: для учебного проекта лучше использовать черновики и ручное подтверждение. |

## Основные задачи реализации

- Сделать симулятор входящих сообщений или подключить Telegram API.
- Реализовать буферизацию пачки сообщений по chat_id.
- Сформировать Style Profile или набор примеров стиля.
- Сгенерировать черновик ответа через LLM.
- Добавить логирование и режимы AUTO_LOW, DRAFT_MEDIUM, BLOCK_HIGH или их аналоги.
- Показать, что рискованные темы не отправляются автоматически.

## Уровни выполнения

| Минимальный уровень | Хорошая работа | Цель максимум | Что сдать |
|---|---|---|---|
| Минимальный зачётный вариант: Telegram/симулятор сообщений -> буфер -> LLM-черновик ответа -> логирование результата. Автоматическая отправка не обязательна. | Добавить Google Sheets/SQLite-лог, Style Profile, несколько режимов ответа: AUTO_LOW, DRAFT_MEDIUM, BLOCK_HIGH. | Сделать полноценный Telegram-автопомощник с Telethon/Pyrogram, задержкой ответа, безопасными режимами, черновиками и ручным подтверждением для рискованных тем. | Архитектура, код или MVP, скриншоты, пример входящих сообщений, сгенерированные черновики, логи, описание ограничений безопасности. |

## Ключевое взаимодействие ВМ и LLM

- Проект может требовать больше стеков: Telegram Client API, Google Sheets, SQLite/Redis, scheduler, LLM API, backend на ВМ.
- Для учебной сдачи достаточно показать контролируемый MVP, где LLM предлагает черновик, а не скрыто подменяет человека.
- Обязательно реализовать или описать фильтр рискованных тем и режим ручного подтверждения.
- Этот вариант считается сложным и может быть принят как полноценный зачётный проект при рабочем прототипе.

## Методические замечания

- Скрытая подмена человека в личной переписке не является целью задания.
- Учебно корректный формат — ассистент, который предлагает черновик и объясняет, почему сообщение безопасное или рискованное.

*Архитектура проекта: Проект 10. Telegram-автопомощник / style assistant*

## 5. Что необходимо сдать

В итоговую сдачу должны входить следующие материалы:

11. Ссылка на GitHub-репозиторий или архив проекта.
12. Файл README.md с описанием проекта, схемой запуска и перечнем зависимостей.
13. Файл .env.example без настоящих ключей.
14. Jupyter Notebook (.ipynb) или исходные файлы приложения/бота.
15. Скриншот созданной ВМ или работающей среды.
16. Скриншот JupyterLab, сайта, бота или n8n workflow.
17. Скриншот или лог успешного обращения к LLM.
18. Краткий отчёт: цель проекта, архитектура, что получилось, какие ограничения есть.
19. Итоговый результат: таблица, отчёт, бот, сайт, RAG-ответы, ревью кода или другой продукт в зависимости от выбранной темы.

**Отдельное требование по безопасности.** Нельзя отправлять настоящие API-ключи, токены Telegram, приватные SSH-ключи или пароли в репозиторий, отчёт, скриншоты и общий чат. Используйте .env.example и переменные окружения.

## 6. Оценивание

| Критерий | Баллы |
|---|---|
| Создание и настройка ВМ, SSH, JupyterLab | 2.0 |
| Корректное подключение LLM и тестовый запрос | 2.0 |
| Реализация выбранного проекта и рабочий результат | 2.5 |
| Осмысленное использование LLM в логике проекта, а не только разовый запрос | 1.5 |
| Аккуратность кода, структура репозитория, .env.example, отсутствие утечки ключей | 1.0 |
| Отчёт, скриншоты, объяснение архитектуры и выводы | 1.0 |
| **Итого** | **10.0** |

### Бонусные элементы

- Рабочий Streamlit/FastAPI-сайт: до +1 балла в рамках округления итоговой оценки.
- Рабочий Telegram-бот или n8n workflow: до +1 балла.
- mini-RAG с источниками и retrieval: до +1.5 балла.
- Стабильный деплой через systemd/Docker, логирование и обработка ошибок: до +1 балла.
- Продуманная архитектура и хороший пользовательский сценарий: до +1 балла.

## 7. Критерии “не принимается”

- Нет работающей ВМ или не показан удалённый запуск среды.
- Нет фактического обращения к LLM.
- LLM используется только для генерации текста в отчёте, но не встроена в проект.
- В репозитории опубликованы настоящие токены, API-ключи или приватные ключи.
- Проект не запускается и не содержит инструкции по воспроизведению.

- Студент не может объяснить, что делает код и как связаны ВМ, Jupyter, backend и LLM.

## 8. Рекомендуемая структура README / отчёта

20. Название проекта.
21. Краткая цель и сценарий использования.
22. Архитектура: какие компоненты используются и как они взаимодействуют.
23. Инструкция запуска.
24. Где именно используется LLM.
25. Примеры входных данных и результатов.
26. Скриншоты работы.
27. Ограничения и возможные улучшения.

```
Пример структуры репозитория:
project_name/
  README.md
  .env.example
  requirements.txt
  notebooks/
    main.ipynb
  src/
    app.py
    llm_client.py
    prompts.py
  data/
    sample.csv
  outputs/
    result.csv
```

**Итоговая цель домашнего задания:** получить не просто установленные библиотеки, а небольшой рабочий AI-проект, где облачная виртуальная машина, Jupyter/Python и LLM взаимодействуют в единой воспроизводимой системе.